

```
#include <stdio.h>
#define MAX_MEMORY 1000
int memory[MAX_MEMORY];
void initializeMemory()
{
    for(int i = 0; i < MAX_MEMORY; i++)
    {
        memory[i] = -1;
    }
}
void displayMemory()
{
    int i, j;

    printf("\nMemory Status:\n");
    for(i = 0; i < MAX_MEMORY; i++)
    {
        if(memory[i] == -1)
        {
            j = i;

            while(j < MAX_MEMORY && memory[j] == -1)
            {
```

```

        while(j < MAX_MEMORY && memory[j] == -1)
        {
            j++;
        }

        printf("Free Block : %d - %d\n", i, j - 1);

        i = j - 1;
    }
}

void allocateMemory(int processId, int size)
{
    int start = -1;
    int blockSize = 0;
    int bestStart = -1;
    int bestSize = MAX_MEMORY;
    for(int i = 0; i < MAX_MEMORY; i++)
    {
        if(memory[i] == -1)
        {
            if(blockSize == 0)
            {

```

```
        if(blockSize == 0)
        {
            start = i;
        }

        blockSize++;
    }
    else
    {
        if(blockSize >= size && blockSize < bestSize)
        {
            bestSize = blockSize;
            bestStart = start;
        }

        blockSize = 0;
    }
}
if(blockSize >= size && blockSize < bestSize)
{
    bestSize = blockSize;
    bestStart = start;
}
```

```

{
    for(int i = bestStart; i < bestStart + size; i++)
    {
        memory[i] = processId;
    }

    printf("\nAllocated Block %d - %d to Process %d\n",
        bestStart,
        bestStart + size - 1,
        processId);
}
else
{
    printf("\nAllocation Failed\n");
}
}

void deallocateMemory(int processId)
{
    for(int i = 0; i < MAX_MEMORY; i++)
    {
        if(memory[i] == processId)
        {
            memory[i] = -1;
        }
    }

    printf("\nMemory released by Process %d\n", processId);
}

int main()
{
    initializeMemory();
    allocateMemory(1, 200);
    allocateMemory(2, 300);
    // ...
}

```

```

    }
    else
    {
        printf("\nAllocation Failed\n");
    }
}

void deallocateMemory(int processId)
{
    for(int i = 0; i < MAX_MEMORY; i++)
    {
        if(memory[i] == processId)
        {
            memory[i] = -1;
        }
    }
    printf("\nMemory released by Process %d\n", processId);
}

int main()
{
    initializeMemory();
    allocateMemory(1, 200);
    allocateMemory(2, 300);
    displayMemory();
    deallocateMemory(1);
    displayMemory();
    allocateMemory(3, 150);
    displayMemory();
    return 0;
}

```

Memory Status:

Free Block : 0 - 999

Allocated Block 0 - 199 to Process 1

Memory Status:

Free Block : 200 - 999

Allocated Block 200 - 499 to Process 2

Memory Status:

Free Block : 500 - 999

Memory released by Process 1

Memory Status:

Free Block : 0 - 199

Free Block : 500 - 999

Allocated Block 500 - 649 to Process 3

Memory Status:

Free Block : 0 - 199

Free Block : 650 - 999

Process exited after 0.08476 seconds with return value 0

Press any key to continue . . . |